

ALGORITMOS Y PROGRAMACIÓN

Lab 6

Programación C

Inventario del héroe · Bestiario del dungeon · Combate contra el jefe

Contenidos

01 Tipos de datos primitivos

02 Variables y declaración

03 printf y scanf

04 Operadores

05 Condicionales

06 Switch

07 Loops

08 Arrays

09 Punteros ★

10 Memoria dinámica

11 Cadenas de texto

12 Estructuras (struct)

13 Listas enlazadas

14 Copia vs. referencia

Tipos de datos primitivos

TIPO	TAMAÑO	EJEMPLO
char	1 byte	'A', 'W'
int	4 bytes	42, -7
float	4 bytes	3.14f
double	8 bytes	3.14159

Casting: para división decimal, forzar tipo antes de operar.
`(float)a / b` → 3.333...

```
int    nivel  = 5;
float  peso   = 3.50f;
char   tipo   = 'W';

/* casting */
int    entero = a / b;           /* 3 */
float  exacto = (float)a / b;    /* 3.33 */
```

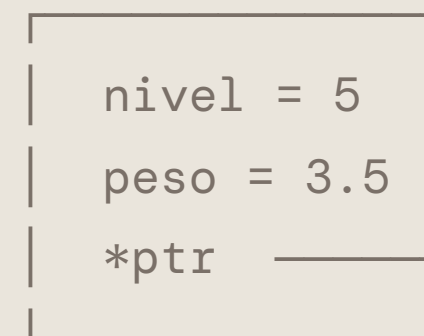
Variables y declaración

```
int    nivel    = 5;  
float  peso     = 3.50f;  
char   tipo     = 'W';  
char   nombre[32] = "Kael";
```

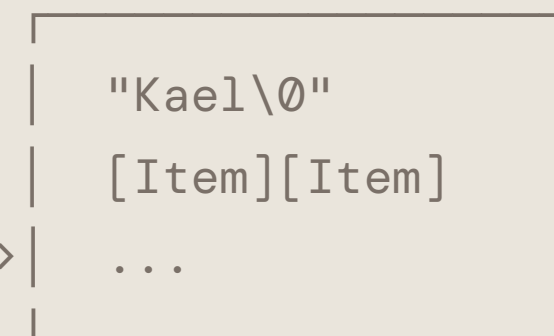
Stack: variables locales — viven mientras dura la función.

Heap: memoria pedida con malloc — persiste hasta free.

Stack



Heap



Los caracteres usan **comillas simples** 'A'.

Las cadenas usan **comillas dobles** "Kael".

printf y scanf

PRINTF — IMPRIMIR

```
printf("Entero:  %d\n", nivel);  
printf("Decimal: %.2f\n", peso);  
printf("Char:    %c\n", tipo);  
printf("Cadena:  %s\n", nombre);  
printf("%-10s", nombre); /* alinear izq */
```

SCANF — LEER DEL TECLADO

```
int opcion;  
scanf("%d", &opcion); /* nota el & */  
  
float peso;  
scanf("%f", &peso);
```

¡El & es obligatorio! scanf necesita la *dirección* de la variable para escribir en ella.

Operadores

ARITMÉTICOS

OP	SIGNIFICADO	RESULTADO
+ - *	Suma, Resta, Mult.	—
/	División entera	10/3 → 3
%	Módulo (resto)	10%3 → 1

LÓGICOS

OP	SIGNIFICADO
&&	AND — ambas verdaderas
	OR — al menos una verdadera
!	NOT — invierte el booleano

COMPARACIÓN

OP	SIGNIFICADO
==	igual a
!=	diferente de
< >	menor / mayor que
<= >=	menor o igual / mayor o igual

En C, **0 es falso** y cualquier otro valor es verdadero.

if / else if / else

```
if (hp >= 50) {  
    printf("Saludable\n");  
} else if (hp > 0) {  
    printf("Herido\n");  
} else {  
    printf("Muerto\n");  
}
```

Evalúa de mayor a menor. Una condición verdadera **detiene** el resto.

PATRÓN: MÍNIMO GARANTIZADO

```
int dmg = atk - def;  
if (dmg < 1) dmg = 1; /* siempre ≥ 1 */
```

PATRÓN: VERIFICAR ANTES DE OPERAR

```
if (b == 0) {  
    printf("Error: div/0\n");  
    return;  
}  
float r = (float)a / b; /* seguro */
```

Switch

```
char tipo = 'W';  
switch (tipo) {  
    case 'W': printf("Arma");      break;  
    case 'A': printf("Armadura"); break;  
    case 'P': printf("Poción");    break;  
    default:  printf("Misc");      break;  
}
```

break es obligatorio. Sin él, la ejecución "cae" al siguiente caso.

¿SWITCH O IF/ELSE?

SITUACIÓN	USAR
Valor exacto ('W' , 1)	switch
Rangos (hp >= 50)	if/else
Con && o	if/else

Loops: for · while · do-while

FOR — ITERACIONES CONOCIDAS

```
for (int i = 0; i < n; i++) {  
    printf("%d\n", arr[i]);  
}
```

DO-WHILE — EJECUTA AL MENOS UNA VEZ

```
do {  
    /* cuerpo primero */  
} while (condicion);
```

WHILE — CONDICIÓN DESCONOCIDA

```
Enemy *cur = head;  
while (cur != NULL) {  
    printf("%s\n", cur->name);  
    cur = cur->next;  
}
```

Para listas enlazadas, **while** es el loop natural — itera hasta NULL.

Arrays

```
int arr[5] = {10, 20, 30, 40, 50};  
/*           0   1   2   3   4   */  
  
arr[2] = 99;           /* modificar */  
printf("%d\n", arr[2]); /* leer */  
  
int n = sizeof(arr) / sizeof(arr[0]);
```

índice: 0 1 2 3 4

arr	10	20	99	40	50
-----	----	----	----	----	----

memoria contigua

Los arrays estáticos **no cambian de tamaño** en ejecución.
Para arrays que crecen → *malloc + realloc*.

09 — PUNTEROS

Un puntero guarda
una **dirección**
de memoria.

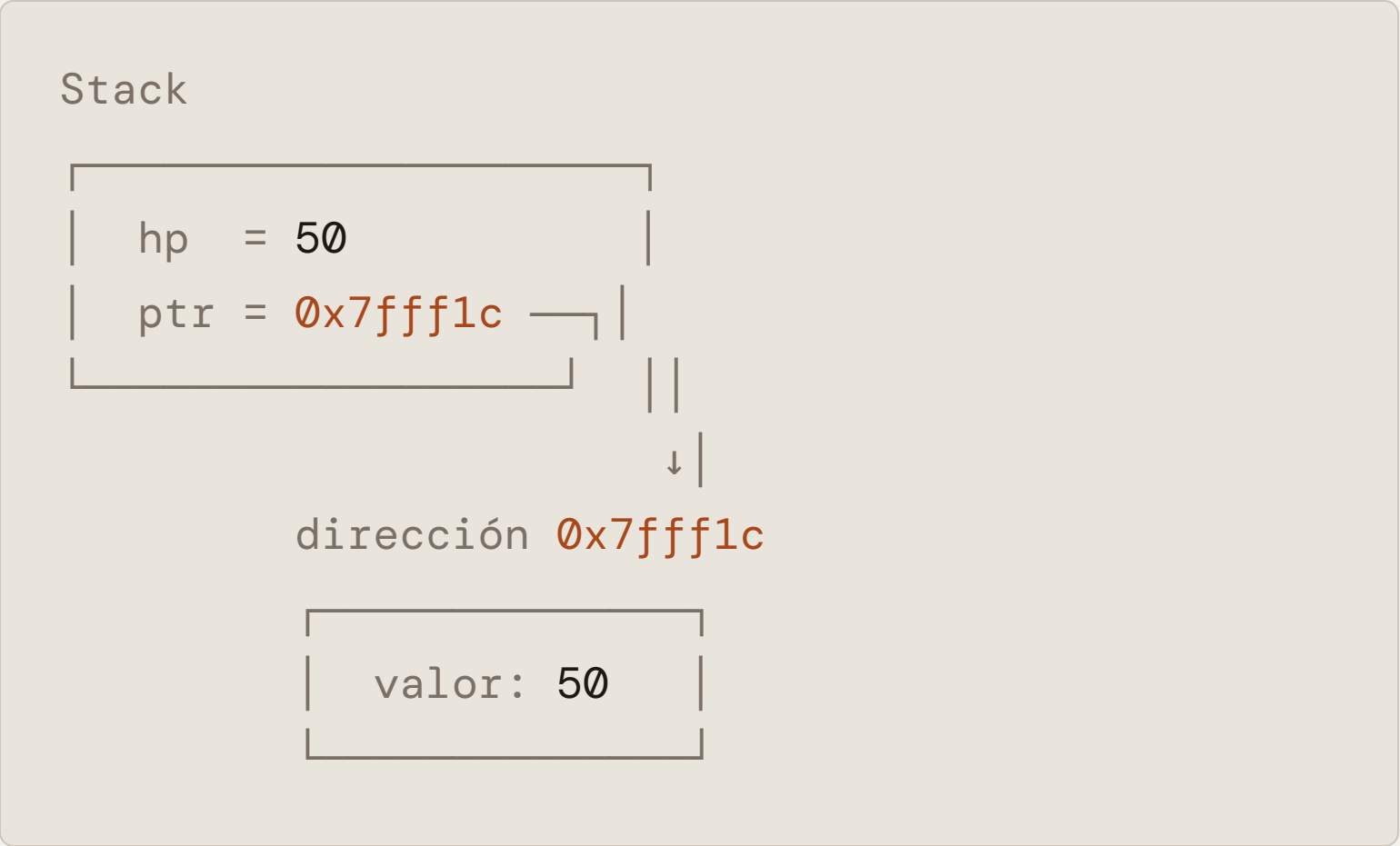
No el valor — la ubicación del valor.

Operadores de punteros

```
int hp = 50;
int *ptr = &hp; /* dirección de hp */

printf("%d\n", *ptr); /* 50 - leer */
*ptr = 30;           /* hp ahora vale 30 */
```

OP	NOMBRE	HACE
&x	Dirección de	Obtiene la dirección de x
*p	Desreferencia	Lee/escribe el valor en p
p->f	Flecha	Campo f de la struct en p



`ptr` contiene una dirección.
`*ptr` es el valor en esa dirección.

El chismoso — struct con punteros

```
typedef struct Person {
    char name[32];
    char message[128];
} Person;

Person teller    = {"Domingo", "Habia una vez un dragon..."};
Person gossip    = {"Jesus",    ""};
Person receiver  = {"Leonardo", ""};

Person *p_teller    = &teller;
Person *p_gossip    = &gossip;
Person *p_receiver  = &receiver;

strcpy(p_gossip->message, p_teller->message);
strcpy(p_receiver->message, p_gossip->message);
```



Versión copia — bloques independientes

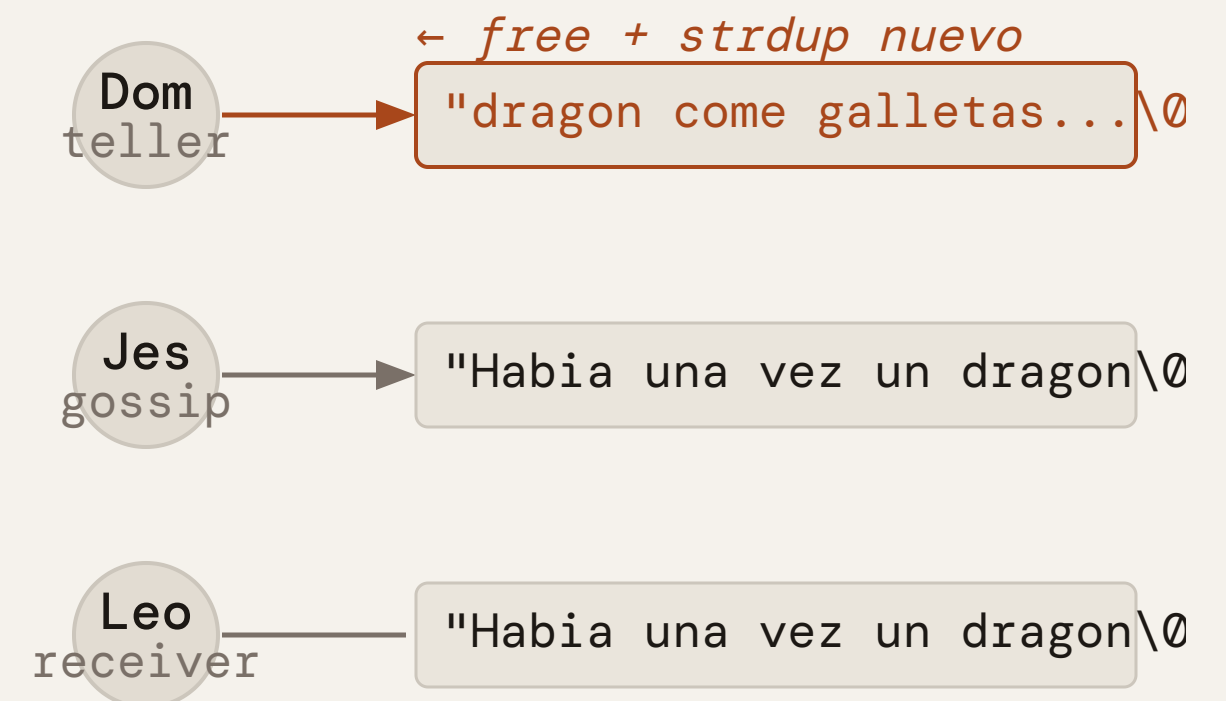
```
typedef struct Person { char name[32]; char *message; } Person;

Person teller = {"Domingo", strdup("Habia una vez un dragon...")};
Person gossip = {"Jesus", strdup(teller.message)}; /* copia */
Person receiver = {"Leonardo", strdup(gossip.message)}; /* copia */

/* Teller cambia su cuento */
free(teller.message);
teller.message = strdup("Ahora el dragon come galletas...");

/* Leonardo sigue con la copia antigua */
free(teller.message); free(gossip.message); free(receiver.message);
```

Cada strdup = bloque propio. Cambiar el de Domingo **no afecta** a Leonardo.



Versión compartida — aliasing

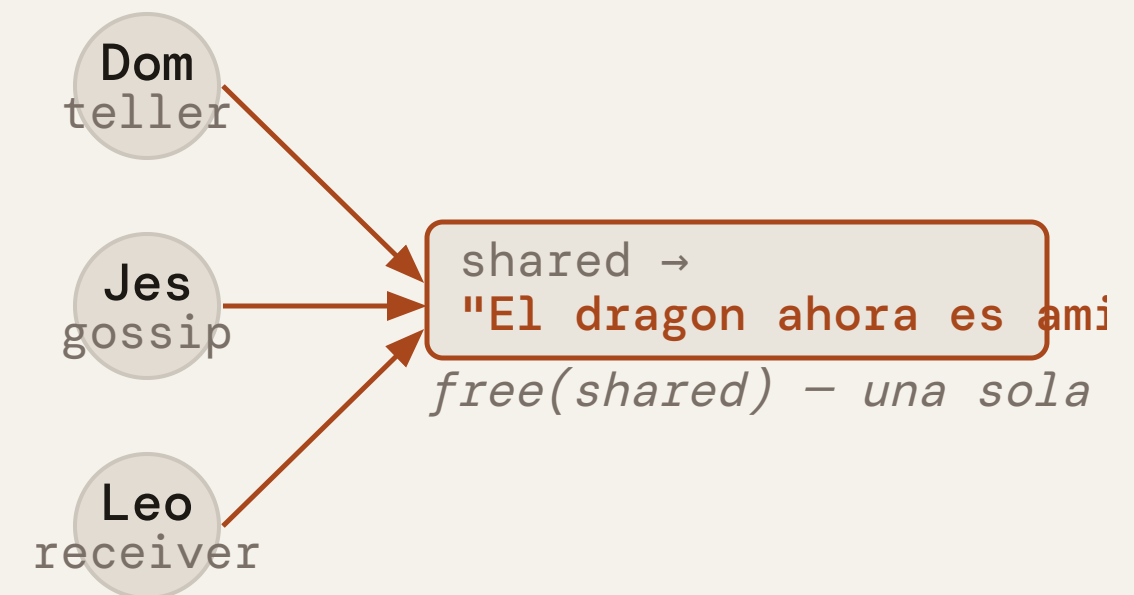
```
char *shared = malloc(128);
strcpy(shared, "Habia una vez un dragon...");

Person teller    = {"Domingo",  shared};
Person gossip    = {"Jesus",     shared}; /* misma dir */
Person receiver  = {"Leonardo", shared}; /* misma dir */

/* Modificar el buffer — todos lo ven */
strcpy(shared, "El dragon ahora es amigo");

free(shared); /* un solo free */
```

Tres punteros, **un solo bloque**. Modificar el buffer — *todos* lo ven. Solo un free.



Puntero a puntero **

Se usa cuando una función necesita **modificar el puntero del llamante**, no solo su contenido.

```
void agregar(Enemy **head, const char *name) {
    Enemy *nuevo = malloc(sizeof(Enemy));
    nuevo->name = strdup(name);
    nuevo->next = *head; /* leer head actual */
    *head = nuevo;      /* actualizar head */
}
```

llamante: head → [nodo A] → NULL

función recibe: head_ptr → head
(Enemy **)

**head = nuevo:*

head → [nuevo] → [nodo A] → NULL

Sin **, la función recibe una **copia** del puntero — los cambios no se propagan.

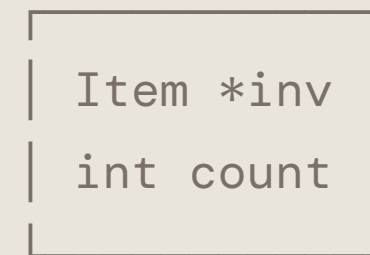
malloc · realloc · free

```
/* 1. Pedir */
Item *inv = malloc(cap * sizeof(Item));
if (!inv) { perror("malloc"); exit(1); }

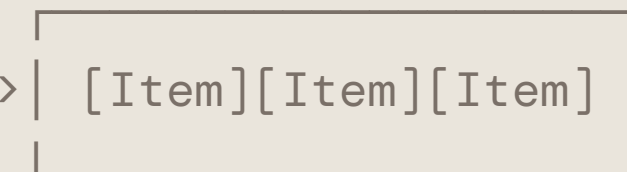
/* 2. Crecer */
cap *= 2;
inv = realloc(inv, cap * sizeof(Item));

/* 3. Liberar campos, luego array */
for (int i = 0; i < n; i++) free(inv[i].name);
free(inv);
inv = NULL;
```

Stack



Heap



realloc → puede
cambiar dirección

Regla de oro: **un free por cada malloc / strdup.**

Primero libera campos puntero, luego la struct.

Strings en C

Un string es un **array de char** terminado en `'\0'`.

```
char nombre[32] = "Kael";  
/* 'K', 'a', 'e', 'l', '\0' */  
  
char *copia = strdup("Espada");  
free(copia); /* obligatorio */
```

FUNCIONES DE <STRING.H>

FUNCIÓN	USO
<code>strlen(s)</code>	Nº de caracteres (sin <code>\0</code>)
<code>strcpy(dst,src)</code>	Copia src en dst
<code>strcmp(a,b)</code>	0 si iguales
<code>strdup(s)</code>	Copia en heap → requiere <code>free</code>

struct

```
typedef struct {  
    char *name;  
    char type;    /* 'W', 'A', 'P' */  
    float weight;  
} Item;  
  
Item it = {strdup("Espada"), 'W', 3.5f};
```

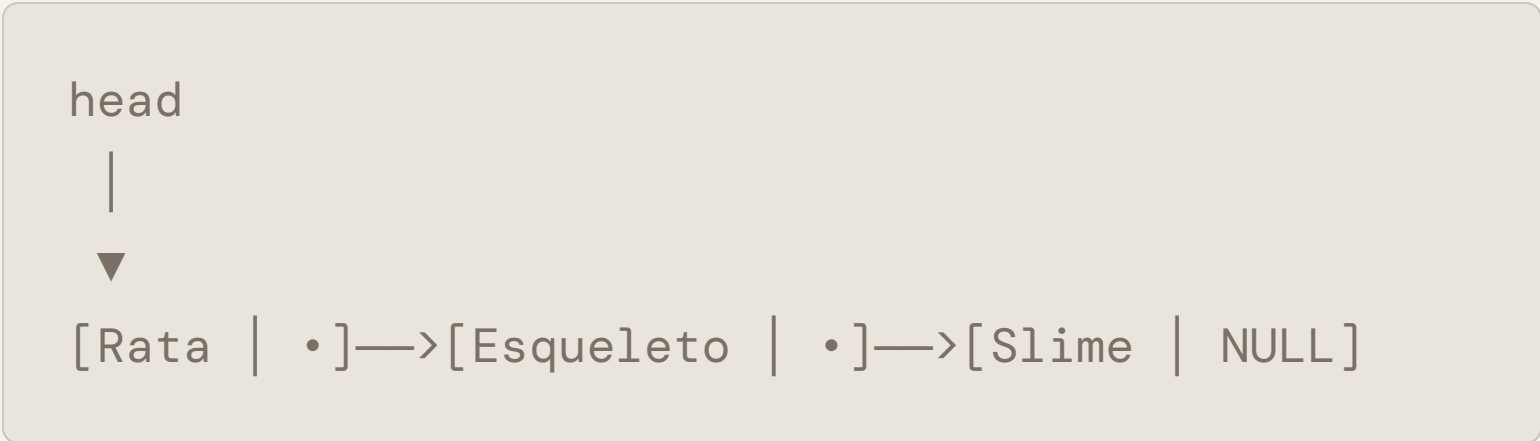
ACCESO A CAMPOS

```
Item it;    it.weight    /* con variable */  
Item *p;    p->weight    /* con puntero */
```

NODO DE LISTA ENLAZADA

```
typedef struct Enemy {  
    char *name;  
    int level;  
    struct Enemy *next; /* se apunta a sí misma */  
} Enemy;
```

Listas enlazadas



```
/* Crear nodo */
Enemy *nuevo = malloc(sizeof(Enemy));
nuevo->name = strdup(name);
nuevo->next = NULL;

/* Liberar toda la lista */
while (head != NULL) {
    Enemy *sig = head->next;
    free(head->name);
    free(head);
    head = sig;
}
```

ARRAYS VS. LISTAS

	ARRAY	LISTA
Acceso por índice	O(1)	O(n)
Insertar al inicio	O(n)	O(1)
Tamaño variable	realloc	Siempre
Memoria contigua	Sí	No

char* en structs

✗ PROBLEMA — COPIA DE PUNTERO

```
Item a; a.name = strdup("Espada");
Item b = a; /* copia el puntero, no el texto */
/* a.name y b.name → mismo bloque */
```

a.name —┐
 ▼
b.name —> [E s p a d a \0] ← un solo bloque

free(a.name) deja b.name apuntando a memoria liberada.
Error grave.

✓ SOLUCIÓN — STRDUP INDEPENDIENTE

```
Item b;
b.name = strdup(a.name); /* nuevo bloque */
b.weight = a.weight;
```

a.name —> [E s p a d a \0] ← bloque A
b.name —> [E s p a d a \0] ← bloque B

Cada struct tiene su propio texto.

Un free por cada strdup.

El inventario del héroe

TEMAS

tipos de datos

struct

malloc / realloc

switch

El héroe guarda items (**armas, armaduras, pociones**) en una mochila que crece automáticamente con `realloc`.

Cuando la mochila se llena, se duplica su capacidad con `realloc` — el array puede cambiar de dirección.

FUNCIONES A IMPLEMENTAR

```
/* Agregar un item al inventario */
void inventory_add(Item **inv, int *count,
                  int *cap, ...);

/* Imprimir con switch por tipo */
void inventory_print(Item *inv, int n);

/* Suma de pesos */
float inventory_total_weight(Item *inv, int n);
```

El bestiario del dungeon

TEMAS

`lista enlazada``malloc / free``struct``loops``if / else`

Registro de enemigos en el dungeon como **lista enlazada**. Cada nodo vive en el heap con su propio nombre en `strdup`.

`bestiary_remove_defeated` usa el patrón **prev + cur** para eliminar nodos mientras recorre la lista sin interrumpirla.

FUNCIONES A IMPLEMENTAR

```
/* Agregar al final */
void bestiary_append(Enemy **head, ...);

/* Insertar en posición N */
void bestiary_insert_at(Enemy **head,
                        int pos, ...);

/* Eliminar nodos defeated == 1 */
void bestiary_remove_defeated(Enemy **head);
```

El combate — Boss Fight

TEMAS

`if / else``aritmética``struct``bucle de juego`

El héroe enfrenta al jefe en un **combate por turnos**. Cada ronda calcula daño, aplica al HP del oponente y verifica si sigue vivo.

Daño mínimo garantizado: `if (dmg < 1) dmg = 1;` — siempre hay al menos 1 punto de daño por turno.

FUNCIONES A IMPLEMENTAR

```
/* atk - def, mínimo 1 */
int  calc_damage(int atk, int def);

/* retorna 1 si hp > 0 */
int  is_alive(Fighter *f);

/* una ronda: héroe ataca, luego jefe */
void combat_round(Fighter *hero,
                  Fighter *boss);
```


LAB 6 — DUNGEON QUEST

¡A programar el dungeon!

Ejercicio 1 — Inventario (malloc/realloc)

Ejercicio 2 — Bestiario (lista enlazada)

Ejercicio 3 — Combate (if/else + aritmética)